

THE COMPLETE ILLUSTRATED REFERENCE

rplib.h

Write C. Skip the repetition.

Every keyword explained in plain words,
with a ready-to-run example and the exact
source code that lives inside it.

99

KEYWORDS

8

CHAPTERS

1

HEADER FILE

```
#include "rplib.h"    // that is all you need
```

Contents

Eight chapters, 99 keywords. Every entry shows what it does, how to use it and the real code behind it.

1. Input / Output	5
2. Math	9
3. Arrays	17
4. Strings	25
5. Loop Helpers	32
6. Matrix	34
7. Search & Sort	39
8. Utilities	45

How rplib.h works

rplib.h is a single header file that gives C short, Python-like keywords. Instead of long standard-library calls you write **Out(...)**, **Sort(a, n)** or **GCD(a, b)**. There is nothing to link: copy the file next to your program and include it.

Quick start

```
#include "rplib.h"

int main(void)
{
    int a[5] = {5, 2, 8, 1, 3};

    Sort(a, 5);
    Print(a, 5); NL();
    Out("Sum = %lld\n", Sum(a, 5));
    return 0;
}
```

What is inside the file

The header starts with an **include guard** (so it can be included twice safely), pulls in the standard headers it wraps, and picks the right system header for delays on Windows or Linux/macOS:

```
#ifndef RPLIB_H
#define RPLIB_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <ctype.h>
#include <time.h>

#ifdef _WIN32
#include <windows.h>
#else
#include <unistd.h>
#endif
```

Two kinds of keywords

FUNCTION	Written as static inline . The compiler can paste the body directly into your code, so there is no speed penalty. Being <i>static</i> , each file gets its own copy, which avoids duplicate-symbol errors.
MACRO	Written with #define . The preprocessor swaps the keyword for real code before compiling. Used for In, Out, NL, Swap, F, R, Repeat and Choose because they need to work with any type or wrap loop syntax.

Compile it

```
gcc main.c -o main -lm
```

The **-lm** flag links the math library (needed by Sqrt, Pow, Round and friends on Linux/macOS). Swap uses **__typeof__**, so use GCC or Clang.

Good to know: exact signatures

Some keywords take more arguments than a quick glance suggests. This book documents the real code, so use these forms:

Keyword	Real form in rplib.h
ReadStr	<code>ReadStr(char *s, int n)</code> - buffer and size
Replace	<code>Replace(char *s, size_t cap, old, nw)</code> - replaces first match only
Transpose	<code>Transpose(a, out, r, c)</code> - writes into a separate output array
Choose	<code>Choose(cond, a, b)</code> - returns a if cond is true, else b
Swap	<code>Swap(x, y)</code> - pass variables, not <code>&x &y</code>
Delay	<code>Delay(seconds)</code> - this is the pause function (there is no <code>Sleep</code>)
TimerStop	<code>TimerStop(start)</code> - pass the value returned by <code>TimerStart()</code>
Max / Min / Abs	Work on double values and return double
Insert / Delete / Unique	Return the new array length - store it in n

CHAPTER 1

Input / Output

Talk to the keyboard and screen in fewer keystrokes.

11 keywords

[In](#) [Out](#) [NL](#) [ReadInt](#) [ReadFloat](#) [ReadLine](#) [ReadStr](#) [PrintInt](#) [PrintFloat](#) [PrintStr](#)
[Pause](#)

In MACRO

In(...)

Shortcut for **scanf**. It is a macro that forwards every argument straight to scanf, so the format string and addresses work exactly as usual.

EXAMPLE

```
int n;  
In("%d", &n);
```

INSIDE THE CODE (from rplib.h)

```
#define In(...)  scanf(__VA_ARGS__)
```

Out MACRO

Out(...)

Shortcut for **printf**. Forwards all arguments to printf, including the format string and any values.

EXAMPLE

```
Out("Value = %d\n", n);
```

INSIDE THE CODE (from rplib.h)

```
#define Out(...)  printf(__VA_ARGS__)
```

NL MACRO

NL()

Prints one newline character using putchar. Handy instead of writing "\n" inside a string.

EXAMPLE

```
Out("Hello"); NL();
```

INSIDE THE CODE (from rplib.h)

```
#define NL()      putchar('\n')
```

ReadInt FUNCTION

```
int ReadInt(void)
```

Reads one integer from the keyboard and **returns it**, so you can write `int n = ReadInt();` instead of declaring the variable and calling scanf separately.

EXAMPLE

```
int n = ReadInt();
```

INSIDE THE CODE (from rplib.h)

```
static inline int ReadInt(void)
{
    int x;
    scanf("%d", &x);
    return x;
}
```

ReadFloat FUNCTION

```
float ReadFloat(void)
```

Reads one decimal number from the keyboard and returns it as a **float**.

EXAMPLE

```
float x = ReadFloat();
```

INSIDE THE CODE (from rplib.h)

```
static inline float ReadFloat(void)
{
    float x;
    scanf("%f", &x);
    return x;
}
```

ReadLine FUNCTION

```
void ReadLine(char *s, int n)
```

Reads a whole line (spaces included) into a buffer of size **n**. It removes the trailing newline and always leaves a valid string, even on end-of-file. Does nothing if the pointer is NULL or n is not positive.

Heads up: If you called ReadInt() just before, a leftover newline is still waiting in the input. Call Pause() once first, or ReadLine will return an empty string.

EXAMPLE

```
char name[50];
ReadLine(name, 50);
```

INSIDE THE CODE (from rplib.h)

```
static inline void ReadLine(char *s, int n)
{
    if (!s || n <= 0) return;

    if (fgets(s, n, stdin))
        s[strcspn(s, "\n")] = '\0';
    else
        s[0] = '\0';
}
```

ReadStr FUNCTION

```
void ReadStr(char *s, int n)
```

A friendly alias of **ReadLine**: reads one line of text into your buffer with a size limit.

Heads up: Takes two arguments (buffer and size), not one.

EXAMPLE

```
char s[100];  
ReadStr(s, 100);
```

INSIDE THE CODE (from rplib.h)

```
static inline void ReadStr(char *s, int n)  
{  
    ReadLine(s, n);  
}
```

PrintInt FUNCTION

```
void PrintInt(int x)
```

Prints an integer with no newline and no extra spaces.

EXAMPLE

```
PrintInt(42);
```

INSIDE THE CODE (from rplib.h)

```
static inline void PrintInt(int x)  
{  
    printf("%d", x);  
}
```

PrintFloat FUNCTION

```
void PrintFloat(double x)
```

Prints a decimal number using the compact **%g** format, so 3.140000 is shown as 3.14.

EXAMPLE

```
PrintFloat(3.14);
```

INSIDE THE CODE (from rplib.h)

```
static inline void PrintFloat(double x)  
{  
    printf("%g", x);  
}
```

PrintStr FUNCTION

```
void PrintStr(const char *s)
```

Prints a string without a newline. A NULL pointer is safely ignored instead of crashing.

EXAMPLE

```
PrintStr("Hello");
```

INSIDE THE CODE (from rplib.h)

```
static inline void PrintStr(const char *s)
{
    if (s) printf("%s", s);
}
```

Pause FUNCTION

```
void Pause(void)
```

Waits for the user to press a key (Enter) by calling getchar. Useful to keep a console window open or to swallow a leftover newline.

EXAMPLE

```
Out("Press Enter...");
Pause();
```

INSIDE THE CODE (from rplib.h)

```
static inline void Pause(void)
{
    getchar();
}
```

CHAPTER 2

Math

Numbers, divisors, primes and special number checks.

20 keywords

Sqrt Pow Abs Max Min Round Floor Ceil Mod Sign GCD LCM Fact IsPrime
IsEven IsOdd IsPerfect IsArmstrong IsPalindrome Digits

Sqrt FUNCTION

double Sqrt(double x)

Square root of x (wrapper for sqrt from math.h). Returns a double.

EXAMPLE

```
double y = Sqrt(25);    /* 5 */
```

INSIDE THE CODE (from rplib.h)

```
static inline double Sqrt(double x)
{
    return sqrt(x);
}
```

Pow FUNCTION

double Pow(double x, double y)

Raises x to the power y (wrapper for pow).

EXAMPLE

```
double y = Pow(2, 5);    /* 32 */
```

INSIDE THE CODE (from rplib.h)

```
static inline double Pow(double x, double y)
{
    return pow(x, y);
}
```

Abs FUNCTION

double Abs(double x)

Absolute value of a number. Works on decimals too because it uses fabs.

Heads up: Returns a double. Print it with %g or cast to int.

EXAMPLE

```
double y = Abs(-10); /* 10 */
```

INSIDE THE CODE (from rplib.h)

```
static inline double Abs(double x)
{
    return fabs(x);
}
```

Max FUNCTION

double Max(double x, double y)

Returns the larger of two numbers using a simple comparison.

Heads up: Returns double. For int arrays use MaxA instead.

EXAMPLE

```
double y = Max(10, 20); /* 20 */
```

INSIDE THE CODE (from rplib.h)

```
static inline double Max(double x, double y)
{
    return x > y ? x : y;
}
```

Min FUNCTION

double Min(double x, double y)

Returns the smaller of two numbers.

EXAMPLE

```
double y = Min(10, 20); /* 10 */
```

INSIDE THE CODE (from rplib.h)

```
static inline double Min(double x, double y)
{
    return x < y ? x : y;
}
```

Round FUNCTION

double Round(double x)

Rounds to the nearest whole number (halves go away from zero). Returns a double.

EXAMPLE

```
double y = Round(3.6); /* 4 */
```

INSIDE THE CODE (from rplib.h)

```
static inline double Round(double x)
{
    return round(x);
}
```

Floor FUNCTION

double Floor(double x)

Rounds down to the nearest whole number.

EXAMPLE

```
double y = Floor(3.9); /* 3 */
```

INSIDE THE CODE (from rplib.h)

```
static inline double Floor(double x)
{
    return floor(x);
}
```

Ceil FUNCTION

double Ceil(double x)

Rounds up to the nearest whole number.

EXAMPLE

```
double y = Ceil(3.1); /* 4 */
```

INSIDE THE CODE (from rplib.h)

```
static inline double Ceil(double x)
{
    return ceil(x);
}
```

Mod FUNCTION

long long Mod(long long x, long long y)

Remainder of x divided by y using the % operator. If y is zero it safely returns 0 instead of crashing.

EXAMPLE

```
long long r = Mod(17, 5); /* 2 */
```

INSIDE THE CODE (from rplib.h)

```
static inline long long Mod(long long x, long long y)
{
    return y ? x % y : 0;
}
```

Sign FUNCTION

int Sign(double x)

Tells you the sign of a number: **1** if positive, **-1** if negative, **0** if zero. The trick is that each comparison gives 1 or 0, and subtracting them gives the sign.

EXAMPLE

```
int s = Sign(-8); /* -1 */
```

INSIDE THE CODE (from rplib.h)

```
static inline int Sign(double x)
{
    return (x > 0) - (x < 0);
}
```

GCD FUNCTION

long long GCD(long long a, long long b)

Greatest common divisor using the **Euclidean algorithm**: keep replacing (a, b) with (b, a mod b) until b becomes 0. Negative inputs are made positive first.

EXAMPLE

```
long long g = GCD(24, 18); /* 6 */
```

INSIDE THE CODE (from rplib.h)

```
static inline long long GCD(long long a, long long b)
{
    if (a < 0) a = -a;
    if (b < 0) b = -b;

    while (b)
    {
        long long t = a % b;
        a = b;
        b = t;
    }

    return a;
}
```

LCM FUNCTION

long long LCM(long long a, long long b)

Least common multiple, computed as $(a / \text{GCD}) \times b$. Dividing first keeps the intermediate number small and avoids overflow. Returns 0 if either input is 0.

EXAMPLE

```
long long l = LCM(12, 18); /* 36 */
```

INSIDE THE CODE (from rplib.h)

```
static inline long long LCM(long long a, long long b)
{
    if (!a || !b) return 0;

    return llabs((a / GCD(a,b)) * b);
}
```

Fact FUNCTION**unsigned long long Fact(unsigned int n)**Factorial $n!$ by multiplying $2 \times 3 \times \dots \times n$. Returns an unsigned long long.**Heads up:** Overflows after 20! (the result no longer fits in 64 bits).**EXAMPLE**

```
unsigned long long f = Fact(5); /* 120 */
```

INSIDE THE CODE (from rplib.h)

```
static inline unsigned long long Fact(unsigned int n)
{
    unsigned long long r = 1;

    for (unsigned int i = 2; i <= n; i++)
        r *= i;

    return r;
}
```

IsPrime FUNCTION**int IsPrime(long long n)**Returns 1 if n is prime, else 0. It rejects numbers below 2 and even numbers, then tests only odd divisors up to the square root (the loop condition $i \leq n/i$ avoids overflow).**EXAMPLE**

```
if (IsPrime(29)) Out("Prime\n");
```

INSIDE THE CODE (from rplib.h)

```
static inline int IsPrime(long long n)
{
    if (n < 2) return 0;

    if (n % 2 == 0)
        return n == 2;

    for (long long i = 3; i <= n/i; i += 2)
        if (n % i == 0)
            return 0;

    return 1;
}
```

IsEven FUNCTION**int IsEven(long long n)**Returns 1 when n divides evenly by 2.**EXAMPLE**

```
if (IsEven(10)) Out("Even\n");
```

INSIDE THE CODE (from rplib.h)

```
static inline int IsEven(long long n)
{
    return n % 2 == 0;
}
```

IsOdd FUNCTION

```
int IsOdd(long long n)
```

Returns 1 when n leaves a remainder when divided by 2.

EXAMPLE

```
if (IsOdd(11)) Out("Odd\n");
```

INSIDE THE CODE (from rplib.h)

```
static inline int IsOdd(long long n)
{
    return n % 2 != 0;
}
```

IsPerfect FUNCTION

```
int IsPerfect(long long n)
```

A perfect number equals the sum of its proper divisors ($6 = 1+2+3$). The loop only walks up to the square root and adds each divisor together with its partner n/i , which is much faster than checking every number.

EXAMPLE

```
if (IsPerfect(28)) Out("Perfect\n");
```

INSIDE THE CODE (from rplib.h)

```
static inline int IsPerfect(long long n)
{
    if (n <= 0) return 0;

    long long sum = 1;

    if (n == 1) return 0;

    for (long long i = 2; i <= n/i; i++)
    {
        if (n % i == 0)
        {
            sum += i;

            if (i != n/i)
                sum += n/i;
        }
    }

    return sum == n;
}
```

IsArmstrong FUNCTION

```
int IsArmstrong(long long n)
```

An Armstrong number equals the sum of its digits each raised to the power of the digit count ($153 = 1^3 + 5^3 + 3^3$). First it counts digits, then sums the powers digit by digit.

EXAMPLE

```
if (IsArmstrong(153)) Out("Armstrong\n");
```

INSIDE THE CODE (from rplib.h)

```
static inline int IsArmstrong(long long n)
{
    if (n < 0) return 0;

    long long t = n;
    long long sum = 0;

    int d = (n == 0) ? 1 : 0;

    while (t)
    {
        d++;
        t /= 10;
    }

    t = n;

    while (t)
    {
        int x = t % 10;

        sum += (long long)llround(pow(x,d));

        t /= 10;
    }

    return sum == n;
}
```

IsPalindrome FUNCTION

```
int IsPalindrome(long long n)
```

Builds the reversed number digit by digit and compares it with the original. A do-while loop is used so that 0 works too. Negative numbers are never palindromes here.

EXAMPLE

```
if (IsPalindrome(1221)) Out("Palindrome\n");
```

INSIDE THE CODE (from rplib.h)

```
static inline int IsPalindrome(long long n)
{
    if (n < 0) return 0;

    long long t = n;
    long long r = 0;

    do
    {
        r = r * 10 + t % 10;
        t /= 10;
    } while (t);

    return r == n;
}
```

Digits FUNCTION

```
int Digits(long long n)
```

Counts decimal digits by dividing by 10 until only one digit remains. The sign is ignored, and 0 counts as 1 digit.

EXAMPLE

```
int d = Digits(12345); /* 5 */
```

INSIDE THE CODE (from rplib.h)

```
static inline int Digits(long long n)
{
    int d = 1;

    if (n < 0)
        n = -n;

    while (n >= 10)
    {
        n /= 10;
        d++;
    }

    return d;
}
```

CHAPTER 3

Arrays

Read, print, search and reshape integer arrays.

20 keywords

Read Print Sum Avg MaxA MinA Sort Rev Swap Find First Last Count
Copy Equal Unique Freq Insert Delete Rotate

Read FUNCTION

```
void Read(int *a, int n)
```

Reads **n** integers from the keyboard into the array, filling **a[0]** to **a[n-1]**.

EXAMPLE

```
int a[5];  
Read(a, 5);
```

INSIDE THE CODE (from rplib.h)

```
static inline void Read(int *a, int n)  
{  
    for (int i = 0; i < n; i++)  
        scanf("%d", &a[i]);  
}
```

Print FUNCTION

```
void Print(const int *a, int n)
```

Prints **n** elements separated by single spaces, with no trailing space and no newline.

EXAMPLE

```
Print(a, n); NL();
```

INSIDE THE CODE (from rplib.h)

```
static inline void Print(const int *a, int n)  
{  
    for (int i = 0; i < n; i++)  
    {  
        if (i) putchar(' ');  
        printf("%d", a[i]);  
    }  
}
```

Sum FUNCTION

```
long long Sum(const int *a, int n)
```

Adds up all n elements. The running total is a long long so big sums do not overflow an int.

EXAMPLE

```
long long s = Sum(a, n);
```

INSIDE THE CODE (from rplib.h)

```
static inline long long Sum(const int *a, int n)
{
    long long s = 0;

    for (int i = 0; i < n; i++)
        s += a[i];

    return s;
}
```

Avg FUNCTION

```
double Avg(const int *a, int n)
```

Average of the array (sum divided by n) as a double. Returns 0 for an empty array.

EXAMPLE

```
double av = Avg(a, n);
```

INSIDE THE CODE (from rplib.h)

```
static inline double Avg(const int *a, int n)
{
    return n > 0 ? (double)Sum(a,n)/n : 0;
}
```

MaxA FUNCTION

```
int MaxA(const int *a, int n)
```

Largest element: starts with a[0] and replaces it whenever a bigger value is found. Returns 0 for an empty array.

EXAMPLE

```
int m = MaxA(a, n);
```

INSIDE THE CODE (from rplib.h)

```
static inline int MaxA(const int *a, int n)
{
    if (n <= 0) return 0;

    int m = a[0];

    for (int i = 1; i < n; i++)
        if (a[i] > m)
            m = a[i];

    return m;
}
```

MinA FUNCTION

```
int MinA(const int *a, int n)
```

Smallest element, using the same scan as MaxA but looking for smaller values.

EXAMPLE

```
int m = MinA(a, n);
```

INSIDE THE CODE (from rplib.h)

```
static inline int MinA(const int *a, int n)
{
    if (n <= 0) return 0;

    int m = a[0];

    for (int i = 1; i < n; i++)
        if (a[i] < m)
            m = a[i];

    return m;
}
```

Sort FUNCTION

```
void Sort(int *a, int n)
```

Sorts the array in ascending order using **bubble sort**: neighbouring elements are swapped when out of order, so the biggest values float to the end on each pass.

Heads up: Simple but slow for large arrays ($O(n^2)$). Use QuickSort or MergeSort for big data.

EXAMPLE

```
int a[5] = {5, 2, 8, 1, 3};
Sort(a, 5); /* 1 2 3 5 8 */
```

INSIDE THE CODE (from rplib.h)

```
static inline void Sort(int *a, int n)
{
    for (int i = 0; i < n-1; i++)
    {
        for (int j = 0; j < n-i-1; j++)
        {
            if (a[j] > a[j+1])
            {
                int t = a[j];

                a[j] = a[j+1];
                a[j+1] = t;
            }
        }
    }
}
```

Rev FUNCTION

```
void Rev(int *a, int n)
```

Reverses the array in place by swapping the first and last elements, then moving both ends inward.

EXAMPLE

```
Rev(a, n);
```

INSIDE THE CODE (from rplib.h)

```
static inline void Rev(int *a, int n)
{
    for (int i = 0, j = n-1; i < j; i++, j--)
    {
        int t = a[i];

        a[i] = a[j];
        a[j] = t;
    }
}
```

Swap MACRO

```
Swap(a, b)
```

Swaps any two variables or array elements of the same type. It is a macro that uses `__typeof__` to create a temporary of the right type, so it works for int, double, char and more. **Pass the variables themselves, not their addresses.**

Heads up: Needs GCC or Clang (`__typeof__`). The internal temporary is named t, so avoid passing a variable that is itself called t.

EXAMPLE

```
int x = 1, y = 2;
Swap(x, y);      /* x=2, y=1 */
Swap(a[0], a[3]); /* swap elements */
```

INSIDE THE CODE (from rplib.h)

```
#define Swap(a,b) \
do { \
    __typeof__(a) t = (a); \
    (a) = (b); \
    (b) = t; \
} while(0)
```

Find FUNCTION

```
int Find(const int *a, int n, int x)
```

Linear search. Returns the index of the first element equal to x, or -1 if it is not present.

EXAMPLE

```
int p = Find(a, n, x);
```

INSIDE THE CODE (from rplib.h)

```
static inline int Find(const int *a, int n, int x)
{
    for (int i = 0; i < n; i++)
        if (a[i] == x)
            return i;

    return -1;
}
```

First FUNCTION

```
int First(const int *a, int n, int x)
```

Index of the first occurrence of x (same as Find).

EXAMPLE

```
int p = First(a, n, x);
```

INSIDE THE CODE (from rplib.h)

```
static inline int First(const int *a, int n, int x)
{
    return Find(a,n,x);
}
```

Last FUNCTION

```
int Last(const int *a, int n, int x)
```

Index of the last occurrence of x, found by scanning from the end. Returns -1 if absent.

EXAMPLE

```
int p = Last(a, n, x);
```

INSIDE THE CODE (from rplib.h)

```
static inline int Last(const int *a, int n, int x)
{
    for (int i = n-1; i >= 0; i--)
        if (a[i] == x)
            return i;

    return -1;
}
```

Count FUNCTION

```
int Count(const int *a, int n, int x)
```

Counts how many elements equal x.

EXAMPLE

```
int c = Count(a, n, x);
```

INSIDE THE CODE (from rplib.h)

```
static inline int Count(const int *a, int n, int x)
{
    int c = 0;

    for (int i = 0; i < n; i++)
        if (a[i] == x)
            c++;

    return c;
}
```

Copy FUNCTION

```
void Copy(int *a, const int *b, int n)
```

Copies *n* integers from the source array *b* into *a*. It uses `memmove`, which stays correct even if the two ranges overlap. **Destination comes first.**

EXAMPLE

```
Copy(dest, src, n);
```

INSIDE THE CODE (from rplib.h)

```
static inline void Copy(int *a, const int *b, int n)
{
    memmove(a,b,n*sizeof(int));
}
```

Equal FUNCTION

```
int Equal(const int *a, const int *b, int n)
```

Returns 1 if both arrays hold the same values in the same order, otherwise 0. Stops at the first difference.

EXAMPLE

```
if (Equal(a,b, n)) Out("Equal\n");
```

INSIDE THE CODE (from rplib.h)

```
static inline int Equal(const int *a, const int *b, int n)
{
    for (int i = 0; i < n; i++)
        if (a[i] != b[i])
            return 0;

    return 1;
}
```

Unique FUNCTION

```
int Unique(int *a, int n)
```

Removes duplicates **in place**, keeping the first occurrence of each value and preserving order. Returns the new length.

EXAMPLE

```
int a[6] = {1,2,2,3,1,4};
int k = Unique(a, 6); /* k=4: 1 2 3 4 */
```

INSIDE THE CODE (from rplib.h)

```
static inline int Unique(int *a, int n)
{
    int k = 0;

    for (int i = 0; i < n; i++)
    {
        if (Find(a,k,a[i]) == -1)
            a[k++] = a[i];
    }

    return k;
}
```

Freq FUNCTION

```
int Freq(const int *a, int n, int x)
```

Frequency of x in the array (an alias for Count).

EXAMPLE

```
int f = Freq(a, n, x);
```

INSIDE THE CODE (from rplib.h)

```
static inline int Freq(const int *a, int n, int x)
{
    return Count(a,n,x);
}
```

Insert FUNCTION

```
int Insert(int *a, int n, int pos, int x)
```

Inserts x at position pos by shifting later elements one step right. Position is clamped into 0..n. Returns the **new length**.

Heads up: Your array must have room for one more element.

EXAMPLE

```
n = Insert(a, n, 2, 99);
```

INSIDE THE CODE (from rplib.h)

```
static inline int Insert(int *a, int n, int pos, int x)
{
    if (pos < 0) pos = 0;
    if (pos > n) pos = n;

    for (int i = n; i > pos; i--)
        a[i] = a[i-1];

    a[pos] = x;

    return n+1;
}
```

Delete FUNCTION

```
int Delete(int *a, int n, int pos)
```

Removes the element at pos by shifting later elements left. An invalid position changes nothing. Returns the new length.

EXAMPLE

```
n = Delete(a, n, 2);
```

INSIDE THE CODE (from rplib.h)

```
static inline int Delete(int *a, int n, int pos)
{
    if (pos < 0 || pos >= n)
        return n;

    for (int i = pos; i < n-1; i++)
        a[i] = a[i+1];

    return n-1;
}
```

Rotate FUNCTION

```
void Rotate(int *a, int n, int k)
```

Rotates the array to the **right** by k positions (negative k rotates left). k is reduced with modulo first, then the last element is moved to the front k times.

EXAMPLE

```
int a[5] = {1,2,3,4,5};  
Rotate(a, 5, 2); /* 4 5 1 2 3 */
```

INSIDE THE CODE (from rplib.h)

```
static inline void Rotate(int *a, int n, int k)  
{  
    if (n <= 0) return;  
  
    k %= n;  
  
    if (k < 0)  
        k += n;  
  
    while (k--)  
    {  
        int t = a[n-1];  
  
        for (int i = n-1; i > 0; i--)  
            a[i] = a[i-1];  
  
        a[0] = t;  
    }  
}
```

CHAPTER 4

Strings

Length, case, search, trim and replace for text.

15 keywords

Len CopyStr Cat Cmp RevStr Upper Lower Trim CountChar FindChar FindStr
Replace IsDigit IsAlpha IsSpace

Len FUNCTION

```
size_t Len(const char *s)
```

Length of a string (characters before the terminating zero). A NULL pointer gives 0.

EXAMPLE

```
size_t n = Len("rplib"); /* 5 */
```

INSIDE THE CODE (from rplib.h)

```
static inline size_t Len(const char *s)
{
    return s ? strlen(s) : 0;
}
```

CopyStr FUNCTION

```
char *CopyStr(char *a, const char *b)
```

Copies string b into buffer a (wrapper for strcpy). Destination first.

Heads up: No size check: make sure dest is big enough.

EXAMPLE

```
CopyStr(dest, src);
```

INSIDE THE CODE (from rplib.h)

```
static inline char *CopyStr(char *a, const char *b)
{
    return strcpy(a, b);
}
```

Cat FUNCTION

```
char *Cat(char *a, const char *b)
```

Appends string b to the end of a (wrapper for strcat).

Heads up: No size check: a must have room for both strings.

EXAMPLE

```
Cat(a, b);
```

INSIDE THE CODE (from rplib.h)

```
static inline char *Cat(char *a, const char *b)
{
    return strcat(a,b);
}
```

Cmp FUNCTION

```
int Cmp(const char *a, const char *b)
```

Compares two strings. Returns 0 if equal, a negative number if a comes first alphabetically, a positive number otherwise.

EXAMPLE

```
if (Cmp(a, b) == 0) Out("Same\n");
```

INSIDE THE CODE (from rplib.h)

```
static inline int Cmp(const char *a, const char *b)
{
    return strcmp(a,b);
}
```

RevStr FUNCTION

```
void RevStr(char *s)
```

Reverses a string in place by swapping characters from both ends.

EXAMPLE

```
char t[] = "abc";
RevStr(t); /* "cba" */
```

INSIDE THE CODE (from rplib.h)

```
static inline void RevStr(char *s)
{
    if (!s) return;

    int i = 0;
    int j = strlen(s)-1;

    while (i < j)
    {
        char t = s[i];

        s[i] = s[j];
        s[j] = t;

        i++;
        j--;
    }
}
```

Upper FUNCTION

```
void Upper(char *s)
```

Converts every letter to uppercase in place.

EXAMPLE

```
Upper(text);
```

INSIDE THE CODE (from rplib.h)

```
static inline void Upper(char *s)
{
    if (!s) return;

    for (; *s; s++)
        *s = toupper((unsigned char)*s);
}
```

Lower FUNCTION

```
void Lower(char *s)
```

Converts every letter to lowercase in place.

EXAMPLE

```
Lower(text);
```

INSIDE THE CODE (from rplib.h)

```
static inline void Lower(char *s)
{
    if (!s) return;

    for (; *s; s++)
        *s = tolower((unsigned char)*s);
}
```

Trim FUNCTION

```
void Trim(char *s)
```

Removes spaces, tabs and newlines from the **start and end** of a string, in place. The text in the middle is untouched.

EXAMPLE

```
char t[] = "  hi  ";
Trim(t); /* "hi" */
```

INSIDE THE CODE (from rplib.h)

```
static inline void Trim(char *s)
{
    if (!s) return;

    char *p = s;

    while (isspace((unsigned char)*p))
        p++;

    if (p != s)
        memmove(s, p, strlen(p)+1);

    int n = strlen(s);

    while (n && isspace((unsigned char)s[n-1]))
        s[--n] = '\0';
}
```

CountChar FUNCTION

```
int CountChar(const char *s, int c)
```

Counts how many times a character appears in a string.

EXAMPLE

```
int n = CountChar(text, 'a');
```

INSIDE THE CODE (from rplib.h)

```
static inline int CountChar(const char *s, int c)
{
    int n = 0;

    if (!s) return 0;

    for (; *s; s++)
        if (*s == c)
            n++;

    return n;
}
```

FindChar FUNCTION

```
int FindChar(const char *s, int c)
```

Index of the first occurrence of a character, or -1 if not found.

EXAMPLE

```
int p = FindChar(text, 'a');
```

INSIDE THE CODE (from rplib.h)

```
static inline int FindChar(const char *s, int c)
{
    if (!s) return -1;

    char *p = strchr(s,c);

    return p ? p-s : -1;
}
```

FindStr FUNCTION

```
int FindStr(const char *s, const char *t)
```

Index where a substring first appears, or -1 if not found.

EXAMPLE

```
int p = FindStr(text, "lib");
```

INSIDE THE CODE (from rplib.h)

```
static inline int FindStr(const char *s, const char *t)
{
    if (!s || !t) return -1;

    char *p = strstr(s,t);

    return p ? p-s : -1;
}
```

Replace FUNCTION

```
int Replace(char *s, size_t cap, const char *old, const char *nw)
```

Replaces the **first** occurrence of *old* with *nw* inside *s*. The **cap** argument is the buffer size; if the new text would not fit, nothing is changed. Returns 1 on success and 0 otherwise.

Heads up: Takes four arguments (string, capacity, old, new) and replaces only one match.

EXAMPLE

```
char t[50] = "old car";
Replace(t, 50, "old", "new"); /* "new car" */
```

INSIDE THE CODE (from rplib.h)

```
static inline int Replace(char *s, size_t cap,
                        const char *old, const char *nw)
{
    if (!s || !old || !nw || !cap || !*old)
        return 0;

    char *p = strstr(s, old);

    if (!p)
        return 0;

    size_t sl = strlen(s);
    size_t ol = strlen(old);
    size_t nl = strlen(nw);

    if (sl-ol+nl+1 > cap)
        return 0;

    memmove(
        p+nl,
        p+ol,
        sl-(p-s)-ol+1
    );

    memcpy(p, nw, nl);

    return 1;
}
```

IsDigit FUNCTION

```
int IsDigit(int c)
```

Returns non-zero if the character is 0-9. The cast to unsigned char keeps it safe for any input.

EXAMPLE

```
if (IsDigit(c)) Out("digit");
```

INSIDE THE CODE (from rplib.h)

```
static inline int IsDigit(int c)
{
    return isdigit((unsigned char)c);
}
```

IsAlpha FUNCTION

```
int IsAlpha(int c)
```

Returns non-zero if the character is a letter.

EXAMPLE

```
if (IsAlpha(c)) Out("letter");
```

INSIDE THE CODE (from rplib.h)

```
static inline int IsAlpha(int c)
{
    return isalpha((unsigned char)c);
}
```

IsSpace FUNCTION

```
int IsSpace(int c)
```

Returns non-zero for space, tab, newline and other whitespace.

EXAMPLE

```
if (IsSpace(c)) Out("space");
```

INSIDE THE CODE (from rplib.h)

```
static inline int IsSpace(int c)
{
    return isspace((unsigned char)c);
}
```

CHAPTER 5

Loop Helpers

Python-style loops and one-line choices.

4 keywords

F **R** **Repeat** **Choose**

F MACRO

F(i, n)

Forward loop: counts *i* from 0 up to *n*-1. Declare *i* yourself before using it.

EXAMPLE

```
int i;
F(i, 5) { Out("%d ", i); } /* 0 1 2 3 4 */
```

INSIDE THE CODE (from rplib.h)

```
#define F(i,n) \
    for ((i)=0; (i)<(n); (i)++)
```

R MACRO

R(i, n)

Reverse loop: counts *i* from *n*-1 down to 0. Declare *i* as a signed int.

EXAMPLE

```
int i;
R(i, 5) { Out("%d ", i); } /* 4 3 2 1 0 */
```

INSIDE THE CODE (from rplib.h)

```
#define R(i,n) \
    for ((i)=(n)-1; (i)>=0; (i)--)
```

Repeat MACRO

Repeat (n)

Repeats the following block *n* times. It creates its own hidden counter, so you do not need a variable.

EXAMPLE

```
Repeat(3) { Out("Hello\n"); }
```

INSIDE THE CODE (from rplib.h)

```
#define Repeat(n) \
    for (int _i=0; _i<(n); _i++)
```

Choose MACRO

Choose(*c*, *a*, *b*)

A readable one-line choice: returns **a** if the condition is true, otherwise **b**. It is the ternary operator in function-like form.

EXAMPLE

```
int big = Choose(x > y, x, y);
```

INSIDE THE CODE (from rplib.h)

```
#define Choose(c,a,b) \
((c) ? (a) : (b))
```

CHAPTER 6

Matrix

2D data stored as flat arrays: add, multiply, transpose.

10 keywords

ReadMat

PrintMat

MatAdd

MatSub

MatMul

Transpose

Trace

Diag

MatEqual

Identity

ReadMat FUNCTION

```
void ReadMat(int *a,int r,int c)
```

Reads an $r \times c$ matrix from the keyboard, row by row. The matrix is stored as a flat array: element (i, j) lives at index $i*c + j$.

EXAMPLE

```
int m[2*3];
ReadMat(m, 2, 3);
```

INSIDE THE CODE (from rplib.h)

```
static inline void ReadMat(int *a,int r,int c)
{
    for (int i=0;i<r;i++)
        for (int j=0;j<c;j++)
            scanf("%d",&a[i*c+j]);
}
```

PrintMat FUNCTION

```
void PrintMat(const int *a,int r,int c)
```

Prints the matrix one row per line with a space after every value.

EXAMPLE

```
PrintMat(m, 2, 3);
```

INSIDE THE CODE (from rplib.h)

```
static inline void PrintMat(const int *a,int r,int c)
{
    for (int i=0;i<r;i++)
    {
        for (int j=0;j<c;j++)
            printf("%d ",a[i*c+j]);

        NL();
    }
}
```

MatAdd FUNCTION

```
void MatAdd(const int *a, const int *b, int *out, int r, int c)
```

Adds two matrices of the same size and stores the result in **out**. Since both are flat arrays, one loop over $r*c$ items is enough.

EXAMPLE

```
MatAdd(a, b, out, r, c);
```

INSIDE THE CODE (from rplib.h)

```
static inline void MatAdd(  
    const int *a,  
    const int *b,  
    int *out,  
    int r,  
    int c)  
{  
    for (int i=0;i<r*c;i++)  
        out[i]=a[i]+b[i];  
}
```

MatSub FUNCTION

```
void MatSub(const int *a, const int *b, int *out, int r, int c)
```

Subtracts matrix b from a, element by element, into out.

EXAMPLE

```
MatSub(a, b, out, r, c);
```

INSIDE THE CODE (from rplib.h)

```
static inline void MatSub(  
    const int *a,  
    const int *b,  
    int *out,  
    int r,  
    int c)  
{  
    for (int i=0;i<r*c;i++)  
        out[i]=a[i]-b[i];  
}
```

MatMul FUNCTION

```
void MatMul(const int *a, const int *b, int *out, int r1, int c1, int c2)
```

Multiplies an $r1 \times c1$ matrix by a $c1 \times c2$ matrix. Each result cell is the sum of (row of a) x (column of b), using three nested loops. Result goes to out ($r1 \times c2$).

EXAMPLE

```
MatMul(a, b, out, 2, 3, 2);
```

INSIDE THE CODE (from rplib.h)

```
static inline void MatMul(  
    const int *a,  
    const int *b,  
    int *out,  
    int r1,  
    int c1,  
    int c2)  
{  
    for (int i=0;i<r1;i++)  
    {  
        for (int j=0;j<c2;j++)  
        {  
            int s=0;  
  
            for (int k=0;k<c1;k++)  
                s += a[i*c1+k] * b[k*c2+j];  
  
            out[i*c2+j]=s;  
        }  
    }  
}
```

Transpose FUNCTION

```
void Transpose(const int *a, int *out, int r, int c)
```

Flips rows and columns: writes the transposed matrix into **out** (size $c \times r$).

Heads up: Takes an output array: Transpose(a, out, r, c).

EXAMPLE

```
Transpose(a, out, r, c);
```

INSIDE THE CODE (from rplib.h)

```
static inline void Transpose(  
    const int *a,  
    int *out,  
    int r,  
    int c)  
{  
    for (int i=0;i<r;i++)  
        for (int j=0;j<c;j++)  
            out[j*r+i]=a[i*c+j];  
}
```

Trace FUNCTION

```
long long Trace(const int *a, int n)
```

Sum of the main diagonal of an $n \times n$ matrix.

EXAMPLE

```
long long t = Trace(m, 3);
```

INSIDE THE CODE (from rplib.h)

```
static inline long long Trace(  
    const int *a,  
    int n)  
{  
    long long s=0;  
  
    for (int i=0;i<n;i++)  
        s += a[i*n+i];  
  
    return s;  
}
```

Diag FUNCTION

```
void Diag(const int *a, int n)
```

Prints the main-diagonal elements separated by spaces.

EXAMPLE

```
Diag(m, 3);
```

INSIDE THE CODE (from rplib.h)

```
static inline void Diag(  
    const int *a,  
    int n)  
{  
    for (int i=0;i<n;i++)  
        printf("%d ", a[i*n+i]);  
}
```

MatEqual FUNCTION

```
int MatEqual(const int *a, const int *b, int r, int c)
```

Returns 1 if two $r \times c$ matrices are identical (reuses Equal on $r*c$ items).

EXAMPLE

```
if (MatEqual(a, b, r, c)) Out("Equal");
```

INSIDE THE CODE (from rplib.h)

```
static inline int MatEqual(  
    const int *a,  
    const int *b,  
    int r,  
    int c)  
{  
    return Equal(a,b,r*c);  
}
```

Identity FUNCTION

```
void Identity(int *a, int n)
```

Fills an $n \times n$ matrix with the identity pattern: 1 on the diagonal, 0 everywhere else.

EXAMPLE

```
int I[9];  
Identity(I, 3);
```

INSIDE THE CODE (from rplib.h)

```
static inline void Identity(  
    int *a,  
    int n)  
{  
    for (int i=0;i<n;i++)  
        for (int j=0;j<n;j++)  
            a[i*n+j]=(i==j);  
}
```

CHAPTER 7

Search & Sort

Classic algorithms ready to call.

10 keywords

[Linear](#)[Binary](#)[Bubble](#)[Select](#)[InsertSort](#)[MergeSort](#)[QuickSort](#)[SecondMax](#)[SecondMin](#)[Missing](#)

Linear FUNCTION

```
int Linear(const int *a, int n, int x)
```

Linear search: checks elements one by one (same as Find). Works on any array.

EXAMPLE

```
int p = Linear(a, n, x);
```

INSIDE THE CODE (from rplib.h)

```
static inline int Linear(  
    const int *a,  
    int n,  
    int x)  
{  
    return Find(a,n,x);  
}
```

Binary FUNCTION

```
int Binary(const int *a, int n, int x)
```

Binary search: repeatedly looks at the middle element and discards half of the array. Much faster than linear search, returns the index or -1.

Heads up: The array must already be sorted in ascending order.

EXAMPLE

```
Sort(a, n);  
int p = Binary(a, n, x);
```

INSIDE THE CODE (from rplib.h)

```
static inline int Binary(  
    const int *a,  
    int n,  
    int x)  
{  
    int l=0;  
    int r=n-1;  
  
    while(l<=r)  
    {  
        int m=l+(r-l)/2;  
  
        if(a[m]==x)  
            return m;  
  
        if(a[m]<x)  
            l=m+1;  
        else  
            r=m-1;  
    }  
  
    return -1;  
}
```

Bubble FUNCTION

```
void Bubble(int *a,int n)
```

Bubble sort (calls Sort, which is bubble sort).

EXAMPLE

```
Bubble(a, n);
```

INSIDE THE CODE (from rplib.h)

```
static inline void Bubble(int *a,int n)  
{  
    Sort(a,n);  
}
```

Select FUNCTION

```
void Select(int *a,int n)
```

Selection sort: for each position, finds the smallest remaining value and swaps it into place.

EXAMPLE

```
Select(a, n);
```

INSIDE THE CODE (from rplib.h)

```
static inline void Select(int *a,int n)
{
    for(int i=0;i<n-1;i++)
    {
        int p=i;

        for(int j=i+1;j<n;j++)
            if(a[j]<a[p])
                p=j;

        if(p!=i)
            Swap(a[i],a[p]);
    }
}
```

InsertSort FUNCTION

```
void InsertSort(int *a,int n)
```

Insertion sort: takes each element and slides it left into its correct place among the already-sorted part, like sorting playing cards in your hand.

EXAMPLE

```
InsertSort(a, n);
```

INSIDE THE CODE (from rplib.h)

```
static inline void InsertSort(int *a,int n)
{
    for(int i=1;i<n;i++)
    {
        int x=a[i];
        int j=i-1;

        while(j>=0 && a[j]>x)
        {
            a[j+1]=a[j];
            j--;
        }

        a[j+1]=x;
    }
}
```

MergeSort FUNCTION

```
void MergeSort(int *a,int n)
```

Bottom-up merge sort: merges sorted blocks of width 1, 2, 4, 8 ... using a temporary buffer until the whole array is sorted. Fast and predictable, $O(n \log n)$.

EXAMPLE

```
MergeSort(a, n);
```

INSIDE THE CODE (from rplib.h)

```
static inline void MergeSort(int *a,int n)
{
    if(n<2) return;

    int *t=malloc(n*sizeof(int));

    if(!t) return;

    for(int w=1;w<n;w*=2)
    {
        for(int l=0;l<n;l+=2*w)
        {
            int m=l+w;
            int r=l+2*w;

            if(m>n) m=n;
            if(r>n) r=n;

            int i=l;
            int j=m;
            int k=l;

            while(i<m && j<r)
                t[k++]=(a[i]<=a[j])?a[i++]:a[j++];

            while(i<m)
                t[k++]=a[i++];

            while(j<r)
                t[k++]=a[j++];
        }

        memcpy(a,t,n*sizeof(int));

        if(w>n/2)
            break;
    }

    free(t);
}
```

QuickSort FUNCTION

```
void QuickSort(int *a,int n)
```

Quicksort: picks the middle value as a pivot, swaps elements so smaller ones go left and larger ones go right, then sorts the two halves recursively. Average $O(n \log n)$.

EXAMPLE

```
QuickSort(a, n);
```

INSIDE THE CODE (from rplib.h)

```
static inline void QuickSort(int *a,int n)
{
    if(n<=1) return;

    int i=0;
    int j=n-1;
    int p=a[n/2];

    while(i<=j)
    {
        while(a[i]<p) i++;
        while(a[j]>p) j--;

        if(i<=j)
        {
            int t=a[i];
            a[i]=a[j];
            a[j]=t;

            i++;
            j--;
        }
    }

    if(j>0)
        QuickSort(a,j+1);

    if(i<n)
        QuickSort(a+i,n-i);
}
```

SecondMax FUNCTION

```
int SecondMax(const int *a, int n)
```

Second-largest **distinct** value. It finds the maximum first, then the biggest value that is not equal to it. Returns 0 if $n < 2$, and the maximum itself if every element is equal.

EXAMPLE

```
int x = SecondMax(a, n);
```

INSIDE THE CODE (from rplib.h)

```
static inline int SecondMax(
    const int *a,
    int n)
{
    if(n<2) return 0;

    int m=MaxA(a,n);
    int found=0;
    int s=0;

    for(int i=0;i<n;i++)
    {
        if(a[i]!=m &&
            (!found || a[i]>s))
        {
            s=a[i];
            found=1;
        }
    }

    return found?s:m;
}
```

SecondMin FUNCTION

```
int SecondMin(const int *a, int n)
```

Second-smallest distinct value, using the same idea as SecondMax.

EXAMPLE

```
int x = SecondMin(a, n);
```

INSIDE THE CODE (from rplib.h)

```
static inline int SecondMin(
    const int *a,
    int n)
{
    if(n<2) return 0;

    int m=MinA(a,n);
    int found=0;
    int s=0;

    for(int i=0;i<n;i++)
    {
        if(a[i]!=m &&
            (!found || a[i]<s))
        {
            s=a[i];
            found=1;
        }
    }

    return found?s:m;
}
```

Missing FUNCTION

```
int Missing(const int *a, int n)
```

Finds the one missing number using the **XOR trick**: XOR of 1..n combined with XOR of all array values cancels every pair and leaves only the missing value.

Heads up: Expects n distinct values taken from 0..n with exactly one of them missing.

EXAMPLE

```
int a[3] = {3, 0, 1};
int x = Missing(a, 3); /* 2 */
```

INSIDE THE CODE (from rplib.h)

```
static inline int Missing(
    const int *a,
    int n)
{
    int x=0;

    for(int i=1;i<=n;i++)
        x^=i;

    for(int i=0;i<n;i++)
        x^=a[i];

    return x;
}
```

CHAPTER 8

Utilities

Screen, time, randomness, memory and timing.

9 keywords

Clear

Delay

Random

Seed

Malloc

Free

TimerStart

TimerStop

Time

Clear FUNCTION

void Clear(void)

Clears the console screen. It runs **cls** on Windows and **clear** on Linux/macOS, chosen at compile time.

EXAMPLE

```
Clear();
```

INSIDE THE CODE (from rplib.h)

```
static inline void Clear(void)
{
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
}
```

Delay FUNCTION

void Delay(unsigned int seconds)

Pauses the program for the given number of **seconds** (Sleep on Windows, sleep elsewhere).

Heads up: Seconds, not milliseconds.

EXAMPLE

```
Delay(2);    /* wait 2 seconds */
```

INSIDE THE CODE (from rplib.h)

```
static inline void Delay(unsigned int seconds)
{
#ifdef _WIN32
    Sleep(seconds*1000);
#else
    sleep(seconds);
#endif
}
```

Random FUNCTION

int Random(int n)

Random integer from **0 to n-1**. Returns 0 if n is not positive.

EXAMPLE

```
Seed();
int x = Random(100);
```

INSIDE THE CODE (from rplib.h)

```
static inline int Random(int n)
{
    if(n<=0) return 0;

    return rand()%n;
}
```

Seed FUNCTION

void Seed(void)

Seeds the random generator with the current time so each run gives different numbers. Call it once at the start.

EXAMPLE

```
Seed();
```

INSIDE THE CODE (from rplib.h)

```
static inline void Seed(void)
{
    srand((unsigned int)time(NULL));
}
```

Malloc FUNCTION

void *Malloc(size_t n)

Thin wrapper for malloc: allocates n bytes of memory.

EXAMPLE

```
int *p = Malloc(10 * sizeof(int));
```

INSIDE THE CODE (from rplib.h)

```
static inline void *Malloc(size_t n)
{
    return malloc(n);
}
```

Free FUNCTION

void Free(void *p)

Releases memory obtained from Malloc (wrapper for free).

EXAMPLE

```
Free(p);
```

INSIDE THE CODE (from rplib.h)

```
static inline void Free(void *p)
{
    free(p);
}
```

TimerStart FUNCTION

clock_t TimerStart(void)

Returns the current processor clock value. Keep it and pass it to TimerStop later.

EXAMPLE

```
clock_t t0 = TimerStart();
```

INSIDE THE CODE (from rplib.h)

```
static inline clock_t TimerStart(void)
{
    return clock();
}
```

TimerStop FUNCTION

double TimerStop(clock_t start)

Given the value from TimerStart, returns the elapsed time in **seconds** as a double.

EXAMPLE

```
clock_t t0 = TimerStart();
Sort(a, n);
Out("%g s\n", TimerStop(t0));
```

INSIDE THE CODE (from rplib.h)

```
static inline double TimerStop(clock_t start)
{
    return (double)(clock()-start)
        / CLOCKS_PER_SEC;
}
```

Time FUNCTION

`time_t Time(void)`

Returns the current calendar time (seconds since 1 Jan 1970).

EXAMPLE

```
time_t now = Time();
```

INSIDE THE CODE (from rplib.h)

```
static inline time_t Time(void)
{
    return time(NULL);
}
```

Quick reference

Every keyword on one sheet, grouped by colour.

Input / Output	In Out NL ReadInt ReadFloat ReadLine ReadStr PrintInt PrintFloat PrintStr Pause
Math	Sqrt Pow Abs Max Min Round Floor Ceil Mod Sign GCD LCM Fact IsPrime IsEven IsOdd IsPerfect IsArmstrong IsPalindrome Digits
Arrays	Read Print Sum Avg MaxA MinA Sort Rev Swap Find First Last Count Copy Equal Unique Freq Insert Delete Rotate
Strings	Len CopyStr Cat Cmp RevStr Upper Lower Trim CountChar FindChar FindStr Replace IsDigit IsAlpha IsSpace
Loop Helpers	F R Repeat Choose
Matrix	ReadMat PrintMat MatAdd MatSub MatMul Transpose Trace Diag MatEqual Identity
Search & Sort	Linear Binary Bubble Select InsertSort MergeSort QuickSort SecondMax SecondMin Missing
Utilities	Clear Delay Random Seed Malloc Free TimerStart TimerStop Time